

Attribution

First things first, the code for this project is largely based on the work done by Johannes Bechberger. The repository that I started off with can be found [here](#). The code he wrote was built around the 6.11/6.12 kernel though, so my project shifted focus from trying to build my own scheduler to learning how the scheduler works, how it interacts with the kernel, and seeing if I could update it to run on the 6.17 kernel.

To that end, I also relied on the scx packages, which can be found [here](#). This package provided more up to date examples of eBPF schedulers utilizing sched_ext, as well as the header files necessary for the scx functions to run.

Motivation

I didn't want to just create a scheduling simulator. That's something I've already done before for other classes, and isn't something I wanted to do again. I sought out to create my own scheduler, that could be used in place of Linux's CFS if I wanted to, that I could run actual tasks on. At first, I was going to try to build a scheduler from scratch based on the like of rt.c, but after briefly considering it, I realized that was a task well beyond my ability. After further investigation, I found eBPF schedulers, which looked like a much more reasonable way to implement a new scheduler for the system.

Installation

I was running my schedulers on Ubuntu 25.10. I was originally trying to get it working on 24.04, but I ran into a number of hurdles, primarily involving missing kernel headers for a number of the functions the schedulers use, such as this:

```
root@jesse-MS-7C02:/home/jesse/minimal-scheduler# ./build.sh
rr.bpf.c:30:12: error: call to undeclared function 'scx_bpf_create_dsq'; ISO C99 and later do not support implicit function declarations [-Wimplicit-function-declaration]
   30 |         return scx_bpf_create_dsq(SHARED_DSQ_ID, -1);
      |                ^
rr.bpf.c:36:28: error: call to undeclared function 'scx_bpf_dsq_nr_queued'; ISO C99 and later do not support implicit function declarations [-Wimplicit-function-declaration]
   36 |         u64 slice = 50000000u / scx_bpf_dsq_nr_queued(SHARED_DSQ_ID);
      |                                ^
rr.bpf.c:37:5: error: call to undeclared function 'scx_bpf_dsq_insert'; ISO C99 and later do not support implicit function declarations [-Wimplicit-function-declaration]
   37 |         scx_bpf_dsq_insert(p, SHARED_DSQ_ID, slice, enq_flags);
      |         ^
rr.bpf.c:43:2: error: call to undeclared function 'scx_bpf_dsq_move_to_local'; ISO C99 and later do not support implicit function declarations [-Wimplicit-function-declaration]
   43 |         scx_bpf_dsq_move_to_local(SHARED_DSQ_ID);
      |         ^
```

If you want to run them yourself, start with

```
sudo apt install -y build-essential cmake cargo rustup clang llvm pkg-config libelf-dev
protobuf-compiler libseccomp-dev libbpf-dev bpftool
```

This should get you most of what you need. You then need to build the [scx](#) from source, and potentially [bpftool](#) if it failed to install. After that, you just need to update build.sh with the path to scx/scheds/include so clang can find common.bpf.h.

Implementation

For every scheduler, they're defined by their `sched_ext_ops` structures, which detail their names, how they're initialized, and how they enqueue and dispatch tasks. It can be much, much more detailed, as can be seen in `scx_lavd`, but for a simple scheduler, that's all you need. All 3 schedulers initialize and dispatch tasks in the same way. To initialize, they call `scx_bpf_create_dsq`, which creates a dispatch queue (DSQ) for queued tasks to wait in. To dispatch, they call `scx_bpf_dsq_move_to_local`, which takes a DSQ's id, and moves the first task in the queue to a CPU to be run. Where they differ is in how the tasks are put into the DSQ. In the RR scheduler, every task is given a time slice, defined as $5\text{ms} / \text{scx_bpf_dsq_nr_queued}$, which means they get a quantum proportional to the amount of tasks in the queue, and inserted into the DSQ by calling `scx_bpf_dsq_insert`, passing the time slice as one of the arguments. For FIFO, we just want the tasks to run until they complete, in the order that they're enqueued, so we give them time slices of -1, which is the equivalent of an infinite amount of time. Lastly, for the priority queue, each task gets a weight, calculated by `scx.weight`, and is inserted into the queue by `scx_bpf_dsq_insert_vtime`, which is an alternative function for enqueueing tasks when they need to have a priority associated with them.

The scripts are just there for ease of use. `build.sh` compiles it with a clang command, `start.sh` registers it with the kernel using `bpftool`, and `stop.sh` removes the `sched_ops` file from the `sched_ext` kernel directory, essentially de-registering it, which kills it.

The primary changes I made to the code was updating it to work on the kernel I'm using. The base code did not include headers from `scx`, and instead relied entirely on the headers created in `vmlinux.h`, which was not working for me. I've also removed a number of redefinitions that were present in the original code since, now that I'm directly accessing the headers, was resulting in redefinition errors. To that end, I also renamed a few instances of `scx` functions to match their actual names, rather than the redefinitions the author had.

Testing

All the schedulers can be run using start.sh, and stopped using stop.sh, like this:

```
jesse@jesse-MS-7C02:~/minimal-scheduler$ sudo ./start.sh prio.bpf.c
[sudo: authenticate] Password:
rm: cannot remove '/sys/fs/bpf/sched_ext/sched_ops': No such file or directory
Registered sched_ext_ops sched_ops map id 194 link id 37
priority_scheduler
jesse@jesse-MS-7C02:~/minimal-scheduler$ sudo ./stop.sh
jesse@jesse-MS-7C02:~/minimal-scheduler$
```

I can confirm that the correct scheduler is running by using the command

```
cat /sys/kernel/sched_ext/state /sys/kernel/sched_ext/*/ops 2>/dev/null
```

Which both says if an extensible scheduler is running, and returns its name if one is.

```
root@jesse-MS-7C02:/home/jesse/minimal-scheduler# ./stop.sh
root@jesse-MS-7C02:/home/jesse/minimal-scheduler# cat /sys/kernel/sched_ext/state /sys/kernel/sched_ext/*/ops 2>/dev/null
disabled
root@jesse-MS-7C02:/home/jesse/minimal-scheduler# ./start.sh
rm: cannot remove '/sys/fs/bpf/sched_ext/sched_ops': No such file or directory
Registered sched_ext_ops sched_ops map id 189 link id 36
rr_scheduler
root@jesse-MS-7C02:/home/jesse/minimal-scheduler# cat /sys/kernel/sched_ext/state /sys/kernel/sched_ext/*/ops 2>/dev/null
enabled
rr_scheduler
root@jesse-MS-7C02:/home/jesse/minimal-scheduler#
```

All tests were done with stress-ng, which can be installed with

```
sudo apt install stress-ng
```

Specifically, I used this command

```
stress-ng --matrix 0 -t 60s --metrics-brief
```

Which floods every cpu in your system with floating point operations for 60s, and returns information about how your system performed, like this:

```
root@jesse-MS-7C02:/home/jesse/minimal-scheduler# stress-ng --matrix 0 -t 60s --metrics-brief
stress-ng: info: [176037] setting to a 1 min run per stressor
stress-ng: info: [176037] dispatching hogs: 24 matrix
stress-ng: info: [176037] note: 24 cpus have scaling governors set to powersave and this may impact performance; setting /sys/devices/system/cpu/cpu*/cpu
freq/scaling_governor to 'performance' may improve performance
stress-ng: metric: [176037] stressor      bogo ops real time  usr time  sys time    bogo ops/s    bogo ops/s
stress-ng: metric: [176037]                (secs)    (secs)    (secs)    (real time) (usr+sys time)
stress-ng: metric: [176037] matrix      2706532    60.00    899.42     0.54    45109.44     3007.38
stress-ng: info: [176037] skipped: 0
stress-ng: info: [176037] passed: 24: matrix (24)
stress-ng: info: [176037] failed: 0
stress-ng: info: [176037] metrics untrustworthy: 0
stress-ng: info: [176037] successful run completed in 1 min
```

The user timer is measured as the total active time across every CPU thread, which is why it's higher than 60s.